

Module: `legacy_billing`

This module provides legacy billing utilities used by older parts of the system. Currently it contains a single function, `process_transaction`, which applies a 5% surcharge to a monetary amount. The implementation dates back over a decade and includes known bugs that can raise exceptions for zero or negative inputs.

Functions

`process_transaction(amount)`

Description:

Converts the supplied *amount* to a floating-point number, validates that it is positive, applies a 5% surcharge, and returns the resulting value.

Parameters: - `amount` (*str/int/float*): The transaction amount. It may be provided as a numeric type or as a string that can be parsed by `float()` (e.g., `"200.50"`, `"1e3"`). Whitespace around the string is tolerated because `float()` strips it.

Returns:

`float` – The original amount multiplied by `1.05` (i.e., a 5% increase).

Examples:

```
# Based on observed runtime behavior
result = process_transaction(100)           # → 105.0
result = process_transaction('200.50')     # → 210.525
result = process_transaction('1e3')         # → 1050.0
```

Edge Cases / Notes: - **Negative values:** If the amount is less than zero, a `ValueError` is raised (`"Transaction amount cannot be negative."`).

`python process_transaction(-5) # raises ValueError process_transaction(' -3.14 ') # raises ValueError` - **Zero value:** Supplying exactly `0` triggers a `ZeroDivisionError` because the function attempts to evaluate `100 / 0`. This is a known hidden bug.

`python process_transaction('0') # raises ZeroDivisionError` - The function does **not** perform any rounding; the returned float preserves the full precision of the calculation.